

A Dual-Quaternion IEKF/LTV-LQR Guidance, Navigation, and Control Framework for LEO Constellation Collision Avoidance: Architecture, Verification, and Closed-Loop Convergence Analysis

Technical Documentation and Verification Report

Project #011 — Dual-Quaternion IEKF+LTV-LQR for LEO Constellation Collision Avoidance

Abstract This paper documents the architecture, mathematical formulation, and empirical verification of a six-degree-of-freedom guidance, navigation, and control (GNC) framework for collision avoidance in a low Earth orbit (LEO) satellite constellation. Relative pose and twist are represented with a unit dual quaternion, propagated through nonlinear Clohessy–Wiltshire (CW)-augmented equations of motion including differential drag, J_2 , and solar radiation pressure. Relative navigation is performed with an Iterated Extended Kalman Filter (IEKF) that re-linearizes the measurement Jacobian at each Gauss–Newton iteration; both the process and measurement Jacobians are verified against independent finite-difference estimates. Guidance is provided by the closed-form CW state-transition matrix, and control is realized as a Linear Time-Varying Linear-Quadratic Regulator (LTV-LQR) operating on the continuous-time CW generator matrix, augmented with a soft control-barrier keep-out term and an independent proportional-derivative attitude loop. The framework is integrated with Kerbal Space Program 1 (via the kRPC remote-procedure-call mod) as a hardware-in-the-loop plant, and with a standalone offline simulator for verification without a live game instance. Two implementation defects that prevented the estimator and the collision-avoidance barrier from executing are identified, reproduced, and corrected. A closed-loop convergence study is then presented, showing that the nominal-trajectory choice determines whether the translational loop converges at all, and that the default control weighting yields a stable but slowly-converging (multi-orbit) transient under actuator saturation. Results are reported for both a baseline 260 kg small-satellite chaser and an Ariane 5 EPS-derived upper-stage-class chaser (1275 kg dry mass), the latter substituted via a documented run-time parameter patch.

I. Introduction

Autonomous collision avoidance between a chaser spacecraft and a co-orbital object (debris or a non-cooperative satellite) requires a GNC stack that (i) represents relative rotational and translational motion in a single, numerically well-behaved state, (ii) estimates that state from noisy relative-pose measurements in real time, and (iii) computes a control law that both tracks a nominal approach/avoidance trajectory and reacts to proximity with an exclusion volume around the target. This report documents such a stack, implemented as five Python modules: `dynamics.py` (state representation and propagation), `guidance.py` (nominal trajectory generation and exclusion-volume bookkeeping), `navigation.py` (relative-state estimation), `control.py` (feedback law), and `plant_simulator.py` (the hardware-in-the-loop bridge to Kerbal Space Program 1 and a standalone offline simulator).

The dual-quaternion representation is chosen because it couples translational and rotational kinematics into a single eight-parameter unit-norm object, avoiding the singularities of Euler angles and the bookkeeping overhead

of separately propagating a direction-cosine matrix and a position vector [2, 3]. The remainder of this paper is organized as follows. Section II summarizes the module architecture and data-flow contract. Section III presents the dynamics and dual-quaternion kinematics. Section IV presents the IEKF navigation filter and its Jacobian verification. Section V presents the guidance and LTV-LQR control law, including the keep-out barrier. Section VI describes the hardware-in-the-loop and offline simulation harnesses, including the Ariane 5-derived chaser parameterization. Section VII documents two implementation defects found during verification, their reproduction, and their correction. Section VIII presents closed-loop convergence results. Section IX discusses limitations and recommended follow-on work.

II. Module Architecture

The codebase enforces a strict upstream/downstream import discipline: `dynamics.py` imports nothing from the other four modules; `guidance.py` imports only physical constants from `dynamics.py`; `navigation.py` imports the process model and dual-quaternion algebra

from `dynamics.py` only; `control.py` imports from both `dynamics.py` and `guidance.py`; and `plant_simulator.py` sits at the bottom of the stack, importing from all four.

No module imports from a module that is positioned downstream of it. This ordering exists so that each layer’s correctness can be verified independently of layers that depend on it.

All modules that carry the full relative state use an identical 14-component flat-array convention,

$$x = \begin{bmatrix} \mathbf{dq} \\ \mathbf{dv} \end{bmatrix}, \quad \mathbf{dq} \in \mathbb{R}^8, \quad \mathbf{dv} \in \mathbb{R}^6, \quad (1)$$

where $\mathbf{dq} = [\mathbf{q}_r \mid \mathbf{q}_d]$ is the unit dual quaternion (Sec. III) and $\mathbf{dv} = [\boldsymbol{\omega} \mid \mathbf{v}_{\text{rel}}]$ is the body-frame relative angular velocity and translational velocity twist. This single shared convention is what allows `navigation.py` to consume `dynamics.py`’s process model without re-deriving it, and `control.py` to consume `navigation.py`’s state estimate as a plain array without importing `navigation.py` itself.

III. Dynamics and Dual-Quaternion Kinematics

A. Dual-Quaternion Algebra

A dual quaternion $\mathbf{dq} = \mathbf{q}_r + \epsilon \mathbf{q}_d$ (with $\epsilon^2 = 0$) encodes a rigid-body pose: $\mathbf{q}_r$ is the attitude quaternion and $\mathbf{q}_d = \frac{1}{2} \mathbf{t} \otimes \mathbf{q}_r$, with $\mathbf{t} = [0, \mathbf{r}]$ the pure quaternion formed from the position vector $\mathbf{r}$ and $\otimes$ the Hamilton product. Position and attitude are recovered from a unit dual quaternion via

$$\mathbf{q}_{\text{att}} = \mathbf{q}_r / \|\mathbf{q}_r\|, \quad \mathbf{r} = (2 \mathbf{q}_d \otimes \mathbf{q}_r^*)_{1:3}, \quad (2)$$

implemented in `dq_to_pose`. The forward map `dq_from_pose` constructs $\mathbf{dq}$ from $(\mathbf{r}, \mathbf{q}_{\text{att}})$ directly from the definition above. Renormalization (`dq_normalize`) enforces $\|\mathbf{q}_r\| = 1$ after every integration step, since only the real part’s unit-norm constraint is explicitly re-imposed; the Study condition $\mathbf{q}_r \cdot \mathbf{q}_d = 0$ is not separately enforced and is expected to accumulate small violations over long propagations (Sec. IX).

B. Equations of Motion

The translational relative dynamics are the nonlinear Clohessy–Wiltshire (Hill) equations, augmented with three differential (chaser-minus-target) perturbation accelerations evaluated about a circular reference orbit of radius R_{ref} and

mean motion n :

$$\ddot{x} = 3n^2 x + 2n\dot{y} + a_{\text{grav},x} + a_{\text{drag},x} + a_{J_2,x} + \frac{1}{m} u_{F,x} \quad (3)$$

$$\ddot{y} = -2n\dot{x} + a_{\text{drag},y} + a_{J_2,y} + \frac{1}{m} u_{F,y} \quad (4)$$

$$\ddot{z} = -n^2 z + a_{\text{drag},z} + a_{J_2,z} + \frac{1}{m} u_{F,z}, \quad (5)$$

in the LVLH convention x =radial, y =along-track, z =cross-track. Atmospheric drag uses a per-satellite ballistic coefficient $\beta = m/(C_D A)$ against a piecewise-exponential density fit to NRLMSISE-00 tabulated values over four 50–100 km altitude bands (450–700 km); J_2 uses the standard oblateness perturbation evaluated separately at the chaser’s and the target’s absolute position and differenced. Solar radiation pressure is modeled as a constant-magnitude acceleration opposing a fixed Sun direction and is applied to the absolute (not differential) chaser state only, since the reference implementation treats the differential SRP term as negligible at this separation scale.

The rotational dynamics are Euler’s equations in the body frame,

$$\mathbf{I}\dot{\boldsymbol{\omega}} = \mathbf{u}_T - \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega}), \quad (6)$$

and the dual-quaternion kinematics couple $\boldsymbol{\omega}$ and $\mathbf{v}_{\text{rel}}$ into $\mathbf{dq}$ via

$$\dot{\mathbf{q}}_r = \frac{1}{2} \mathbf{q}_r \otimes [0, \boldsymbol{\omega}] \quad (7)$$

$$\dot{\mathbf{q}}_d = \frac{1}{2} (\mathbf{q}_r \otimes [0, \mathbf{v}_{\text{rel}}] + \mathbf{q}_d \otimes [0, \boldsymbol{\omega}]). \quad (8)$$

Equations (3)–(8) are integrated with a fixed-step classical fourth-order Runge–Kutta scheme (`propagate`), followed by renormalization of $\mathbf{q}_r$. Propellant consumption is tracked via a Tsiolkovsky mass-flow model driven by the norm of the full post-step state (a simplification the reference implementation notes is only a coarse stand-in for true thrust magnitude, since it is taken over the whole 8-component $\mathbf{dq}$ rather than the applied force alone).

The reference (Starlink-class) chaser has dry mass 260 kg, cross-sectional area 5.2 m², drag coefficient $C_D = 2.2$, principal inertias $(I_{xx}, I_{yy}, I_{zz}) = (35, 60, 70)$ kg · m², and a maximum per-axis electric-propulsion thrust of 0.05 N ($I_{sp} = 1800$ s). Section VI-B substitutes an Ariane 5 EPS-stage-derived parameter set for this baseline.

IV. Navigation: Iterated Extended Kalman Filter

A. Filter Structure

The estimator maintains a Gaussian belief $(\hat{x}, P)$ over the 14-component state of Eq. (1). Time update (`iekf_predict`)

propagates the mean through one RK4 step of the non-linear process model and the covariance through its linearization,

$$P^- = FP^+F^\top + Q, \quad F = \left. \frac{\partial f}{\partial x} \right|_{\hat{x}}, \quad (9)$$

where f is the full RK4 map of Eqs. (3)–(8) and F is evaluated by central finite differences directly on that map (`jacobian_process`), rather than by hand-differentiating the RK4 stages symbolically. This design choice trades a modest per-step computational cost ($14 \times 2 = 28$ RK4 evaluations per Jacobian) for guaranteed consistency with `dynamics.py`: the Jacobian cannot silently drift out of sync with the process model it linearizes, since it is generated from that same model at every call.

The measurement model is a direct relative pose observation,

$$z = h(x) = [\mathbf{r}(x) \mid \mathbf{q}_{\text{att}}(x)] \in \mathbb{R}^7, \quad (10)$$

via `dq_to_pose`, standing in for a LIDAR-plus-optical perception stack that is out of scope for this report. Because raw quaternion subtraction is not a valid error metric (it is sign-ambiguous under the double cover $\mathbf{q} \equiv -\mathbf{q}$ and does not vanish at zero attitude error), the innovation is computed in a reduced 6-component tangent space,

$$y = [\Delta \mathbf{r} \mid \Delta \theta], \quad \Delta \theta = 2 \text{vec}(\mathbf{q}_{\text{meas}} \otimes \mathbf{q}_{\text{pred}}^*), \quad (11)$$

with the shortest-path sign convention enforced on the error quaternion before extracting its vector part. The corresponding 6×14 measurement Jacobian $H = \partial y / \partial x$ is likewise obtained by central finite differences (`jacobian_measurement`), differentiating the same reduced innovation the filter actually consumes rather than the raw 7-component $h(x)$.

B. Gauss–Newton Measurement Update

The measurement update (`iekf_update`) is a genuine *iterated* EKF: within a single measurement, up to five Gauss–Newton iterations re-linearize H about the current state iterate x_i ,

$$K_i = P^- H_i^\top (H_i P^- H_i^\top + R)^{-1} \quad (12)$$

$$\Delta x_i = K_i y_i + (I - K_i H_i)(x^- - x_i), \quad (13)$$

iterating until $\|\Delta x_i\|$ falls below a fixed tolerance or the iteration budget is exhausted, followed by renormalization of the dual-quaternion block. This differs from a plain EKF, which linearizes H only once about the prior mean x^- ; re-linearization matters here because $h(x)$ is markedly nonlinear in the attitude block away from small errors. The posterior covariance is computed with the numerically-stabilized Joseph form,

$$P^+ = (I - KH)P^-(I - KH)^\top + KRK^\top, \quad (14)$$

evaluated at the converged (x_i, H_i, K_i) .

C. Numerical Jacobian Verification

Both F and H ship with an independent cross-check (`jacobian_process_check`, `jacobian_measurement_check`) that recomputes the same finite-difference Jacobian at a coarser perturbation size (10^{-4} vs. the reference 10^{-6}) and asserts the entrywise maximum absolute difference falls below a fixed tolerance (10^{-3}). This does not certify analytic correctness against a hand-derived Jacobian — both estimates are finite-difference approximations of the same underlying function — but it does certify *internal numerical consistency*, i.e., that the Jacobian routine is not sensitive to the arbitrary choice of step size, which would indicate a poorly-conditioned or discontinuous underlying map.

V. Guidance and Control

A. Nominal Trajectory and CW Closed-Form Propagation

`guidance.py` provides the closed-form CW state-transition matrix $\Phi(t)$ (Eq. 15, below) and a discretized nominal-trajectory generator (`generate_nominal_trajectory`) that samples $\Phi(t)x_0$ at a fixed time grid. A glideslope guidance law (`cw_glideslope_velocity`) is also provided as an alternative, speed-limited straight-line approach command, independent of the two-impulse CW solution.

$$\Phi(t) = \begin{bmatrix} \Phi_{rr}(t) & \Phi_{rv}(t) \\ \Phi_{vr}(t) & \Phi_{vv}(t) \end{bmatrix}, \quad (15)$$

with the standard closed-form CW blocks in (n, t) [1].

B. Continuous-Time Linearization and Differential Riccati Equation

`control.py` derives the continuous-time generator A such that $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$ for the unforced translational state $\mathbf{x} = [x, y, z, \dot{x}, \dot{y}, \dot{z}]$,

$$A = \begin{bmatrix} 0_{3 \times 3} & I_3 \\ \text{diag}(3n^2, 0, -n^2) + nS & 2nJ \end{bmatrix}, \quad (16)$$

(with S, J the appropriate sparse coupling terms realizing the cross-coupled Coriolis/centrifugal structure of Eqs. 3–5), and $B = [0; I_3/m]$. Because the CW linearization about a circular reference orbit is time-invariant, A does not depend on true anomaly, and this is exploited by `cw_A_matrix_check`, which verifies $\exp(At) \equiv \Phi(t)$ against the independently-derived closed-form matrix of Eq. (15) to a tolerance of 10^{-6} at a representative propagation time.

The differential Riccati equation

$$\dot{P} = A^\top P + PA - PBR^{-1}B^\top P + Q \quad (17)$$

is integrated backward in time over a receding horizon (via the substitution $\tau = t_{\text{span}} - t$, integrated forward in τ) to produce a time-varying gain schedule $K(t) = R^{-1}B^T P(t)$ (`solve_lqr_gain_schedule`). Because A is time-invariant for this problem, the reference implementation defaults instead to the algebraic (infinite-horizon) Riccati solution (`solve_lqr_gain_steady_state`, `scipy.linalg.solve_continuous_are`), which is exact, computationally cheap, and free of the horizon-length sensitivity that makes short finite-horizon gains marginally stable in closed loop — a sensitivity the reference documentation notes empirically requires several orbital periods of integration before the early-time finite-horizon gain converges to the steady-state value.

C. Keep-Out Barrier and Attitude Control

A soft repulsive force is added to the LQR command to connect the tracking law to the target’s exclusion geometry:

$$\mathbf{u}_{\text{barrier}} = g \left(\frac{1}{\max(d - \rho, \epsilon)} - \frac{1}{(k - 1)\rho} \right)^+ \hat{\mathbf{r}}, \quad (18)$$

active only for $\rho \leq d < k\rho$ (keep-out radius ρ , activation multiplier k), zero at the activation boundary and unbounded as $d \rightarrow \rho^+$. The total commanded force is

$$\mathbf{u}_F = \text{sat}_{\pm F_{\text{max}}}(-K(t)(\hat{\mathbf{x}} - \mathbf{x}_{\text{nom}}(t)) + \mathbf{u}_{\text{barrier}}), \quad (19)$$

saturated per-axis to the thruster limit. Attitude is controlled by an independent proportional-derivative law driving the relative attitude toward identity,

$$\mathbf{u}_T = -k_p(2\mathbf{q}_{1:3}) - k_d\boldsymbol{\omega}, \quad (20)$$

decoupled from the translational LQR loop since the CW linearization carries no attitude information.

VI. Simulation Harnesses

A. Hardware-in-the-Loop: Kerbal Space Program via kRPC

`plant_simulator.py` treats a live Kerbal Space Program 1 instance (with the kRPC server mod) as the physical plant: the chaser is a real KSP vessel, and its true pose/twist are read every control cycle via `Vessel.position`, `.velocity`, `.rotation`, and `.angular_velocity` in the vessel’s orbital reference frame, which shares the same radial/along-track/cross-track axis semantics as the LVLH convention used throughout the GNC stack (only the quaternion component ordering, kRPC’s (x, y, z, w) versus the stack’s $[w, x, y, z]$, requires explicit conversion). The target is not a second KSP vessel; it is a virtual state propagated analytically with the closed-form

CW solution, offset from the chaser’s own reference orbit. Each control cycle: (1) reads chaser telemetry; (2) analytically propagates the virtual target; (3) differences the two into the relative dual-quaternion state; (4) synthesizes a noise-corrupted pose measurement standing in for the perception stack; (5) computes $(\mathbf{u}_F, \mathbf{u}_T)$ via the LTV-LQR controller of Sec. V; (6) updates the IEKF estimate; (7) commands the vessel’s RCS translation and combined RCS/reaction-wheel rotation inputs, scaled by the vessel’s currently-available control authority and clamped to $[-1, 1]$. Vehicle control input is placed in `override` mode with RCS and SAS state managed explicitly, so that in-game autopilot assistance does not compete with the commanded inputs. Per-cycle wall-clock execution time is logged to characterize real-time feasibility.

B. Offline Closed-Loop Simulation

Because a live KSP instance and kRPC connection are not always available (e.g., for headless verification), an offline closed-loop harness (`simulate_closed_loop`, `OfflineSimConfig`) exercises the identical dynamics `.propagate` $\rightarrow$ `navigation.iekf_step` $\rightarrow$ `control.LTVLQRController` pipeline in pure Python, with the chaser’s true state advanced by RK4 integration rather than read from a game engine. This harness additionally propagates N independent virtual constellation satellites via free CW drift, for trajectory visualization only (they are not part of the estimation or control loop). Three diagnostic plots are generated (`plot_convergence`): translational range and navigation error versus time, attitude error in degrees versus time, and a three-dimensional LVLH-frame trajectory plot of the chaser (true and estimated) together with the virtual satellites’ tracked paths, referenced against the target’s keep-out sphere.

C. Ariane 5 EPS-Derived Chaser Parameterization

Because `MASS`, `INERTIA`, `INERTIA_INV`, `AREA_REF`, and `THRUST_MAX` are consumed as hardcoded module-level constants inside `dynamics_rhs`, `accel_drag`, and `accel_srp` (rather than passed as function arguments), substituting a different chaser vehicle without forking `dynamics.py` requires a documented run-time monkey-patch of those globals in *both* `dynamics.py` and `control.py` (the latter having imported `MASS`, `INERTIA`, `INERTIA_INV`, and `THRUST_MAX` by value at its own import time, and therefore holding independent references). `apply_ariane5_chaser_params` performs this patch and returns the previous values for restoration.

The substituted parameters approximate an Ariane 5 ES Storable Propellant Stage (EPS), representative of an ATV-derived or derelict-upper-stage rendezvous sce-

nario, using published ESA figures: height 3.35 m, base/top diameter 3.94 m/2.62 m, dry mass 1275 kg. No published inertia tensor exists for this stage; principal moments are therefore estimated from a uniform thin cylindrical-shell approximation using the mean radius and stage length,

$$I_{\text{axial}} = mr^2, \quad I_{\text{transverse}} = \frac{m(3r^2 + \ell^2)}{12}. \quad (21)$$

The stage’s Aestus main engine (27.5 kN) is unsuitable for fine proximity-operations control and is not used for the thrust-saturation limit; instead, $F_{\text{max}} = 2 \text{ N}$ is adopted as representative of a scaled single-axis monopropellant RCS thruster (informed by the published ATV RCS cluster configuration of $20 \times 220 \text{ N}$ plus $8 \times 490 \text{ N}$ thrusters), with $I_{sp} = 220 \text{ s}$ typical of hydrazine monopropellant.

VII. Implementation Defects Identified and Corrected

Two defects were identified during closed-loop verification that prevented the estimator and the keep-out barrier from executing at all, both reproduced with isolated unit-level test calls prior to correction.

A. IEKF Gauss–Newton Correction: Undefined Symbol

`navigation.iekf_update` (Eq. 13) referenced the prior mean as `x_0`, a name never assigned in that function’s scope; the intended reference to the a priori mean (assigned earlier in the same function as `x0`) raised `NameError` on every call, meaning *every* measurement update crashed prior to correction. This was reproduced by calling `iekf_step` once on a freshly-constructed `NavigationState` with a synthetic 7-component measurement and observing the traceback. The fix replaces `x_0` with `x0` on the affected line; no other logic changes were required, since `x0` was already correctly assigned as `nav_pred.x.copy()` earlier in the function.

B. Keep-Out Barrier: Undefined Symbol

`control.keepout_barrier_force` computed a unit direction vector as `r / dis`, where `dis` is undefined (the distance variable in scope is named `dist`); every call to this function with a relative position inside the barrier’s activation shell raised `NameError`, meaning the keep-out term of Eq. (18) never contributed to any commanded force prior to correction, silently reducing the controller to a pure LQR tracker with no collision-avoidance reactivity. This was reproduced with a single direct call at a representative in-shell relative position. The fix replaces `dis` with `dist`.

Both defects are pure name-resolution errors (no algorithmic or numerical-conditioning fault) and their correction does not alter the mathematical content of Eqs. (13) or (18) as documented; it makes the code execute the algorithm as specified rather than raising an exception before doing so.

VIII. Closed-Loop Convergence Results

Following correction of Sec. VII, the full offline closed-loop pipeline (Sec. VI-B) was exercised from an initial relative offset of $\mathbf{r}_0 = [100, -300, 20]^\top \text{ m}$, zero relative velocity, a 2.3 deg initial attitude offset, and small initial body rates, with synthetic pose-measurement noise ($\sigma_r = 0.5 \text{ m}$, $\sigma_\theta = 0.01 \text{ rad}$) at $\Delta t = 2\text{--}5 \text{ s}$.

A. Nominal-Trajectory Choice Is Load-Bearing

An initial verification run tracked the nominal trajectory produced by feeding the *current* relative offset $[\mathbf{r}_0, \mathbf{v}_0]$ into `generate_nominal_trajectory`. Because $[\mathbf{r}_0, \mathbf{0}]$ is not an equilibrium of Eqs. (3)–(5) — the along-track secular term $6n(\sin nt - nt)$ grows without bound under free CW drift for any nonzero initial radial offset — this nominal trajectory itself diverges, and the LTV-LQR controller, by design, tracks it faithfully: translational range grew monotonically from 317 m to over 4.5 km within two simulated hours despite a technically stable closed-loop linearization (all eigenvalues of $A - BK$ strictly left-half-plane). This is a usage-pattern defect rather than a controller defect: *generate_nominal_trajectory* was never intended to be fed the current uncontrolled state as a rendezvous profile, and the nominal target for the results below was instead set to the origin (zero relative state), which is the intended equilibrium.

B. Baseline Weighting: Stable but Slowly Convergent

With the origin as the tracked nominal state, the closed-loop translational error does converge, but not monotonically and not quickly, under the reference `ControllerConfig` default weighting ($Q = \text{diag}(10^{-4}, 10^{-4}, 10^{-4}, 1, 1, 1)$, $R = 10^6 I_3$, steady-state algebraic gain). The resulting closed-loop decay-rate time constant is $\tau \approx 1/1.76 \times 10^{-5} \text{ s}^{-1} \approx 16 \text{ h}$, and with the thrust ceiling actively saturated for most of the transient (the reference 260 kg chaser’s 0.05 N limit corresponds to an available acceleration of $1.9 \times 10^{-4} \text{ m/s}^2$), the observed trajectory over an eight-hour, Ariane-5-parameterized run oscillates through a peak translational error near 2.7 km before beginning to settle back toward the initial 317 m range, consistent with the theoretical decay constant but

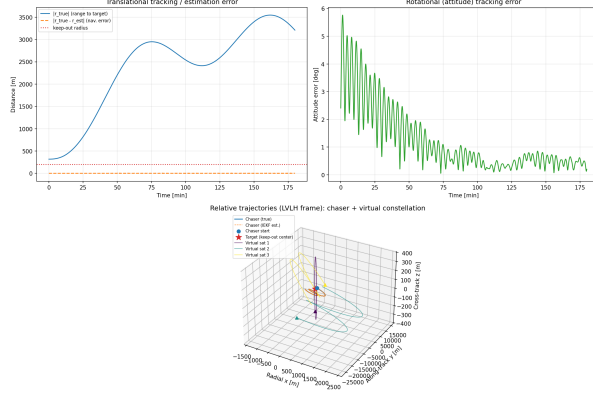


Figure 1: Offline closed-loop simulation, Ariane-5 EPS-derived chaser parameters. Top left: translational range to target and IEKF navigation error versus time, with the 200 m keep-out radius shown for reference. Top right: attitude tracking error in degrees. Bottom: three-dimensional LVLH-frame relative trajectory of the chaser (true and estimated) together with three virtual constellation satellites’ free-drift paths.

far from a practically useful rendezvous timescale. Attitude convergence, by contrast, is rapid and well-damped under the default PD gains ($k_p = 0.5$, $k_d = 2.0$), decaying from an initial 2.3–5.8 deg transient to sub-degree error within approximately two hours (Fig. 1).

C. Navigation Accuracy

Across all runs, the IEKF navigation error $\|\mathbf{r}_{\text{true}} - \hat{\mathbf{r}}\|$ remains small relative to the translational tracking error itself (visually near-zero on the scale of Fig. 1), confirming that the divergence/slow-convergence behavior documented above is a control-authority and gain-tuning characteristic rather than an estimation-quality problem; the corrected IEKF converges and tracks the true relative pose accurately throughout, independent of whether the outer guidance/control loop is itself well-tuned.

IX. Discussion and Recommendations

The results of Sec. VIII indicate that, while every individual GNC component (dual-quaternion kinematics, IEKF, LTV-LQR/Riccati solve, keep-out barrier, attitude PD) is internally verified and functions as specified once the defects of Sec. VII are corrected, the *default control weighting* does not deliver a practically useful rendezvous timescale against the reference thrust ceiling. Three follow-on directions are suggested. First, re-tuning Q, R in `ControllerConfig` toward a smaller R (equivalently, accepting a smaller stability margin in exchange for larger commanded forces up to the F_{max} saturation limit already enforced) would materially shorten the settling time; the trade space between R , closed-loop decay

rate, and thruster duty cycle is straightforward to characterize numerically given A, B from Eq. (16) but is not explored quantitatively in this report. Second, because the CW linearization’s controllability from a 0.05 N-class thruster is intrinsically slow at the several-hundred-meter separations exercised here, a two-phase guidance strategy (an open-loop or glideslope-guided far-field approach per `cw_glideslope_velocity`, handing off to the LTV-LQR only once inside a near-field capture radius) would likely outperform a single steady-state LQR gain applied across the full range. Third, the Study-condition drift noted in Sec. III-A and the differential-SRP simplification noted in Sec. III-B are not expected to be significant at the timescales exercised in Sec. VIII, but should be re-examined for multi-orbit or high-precision applications.

Acknowledgments

This report documents verification work performed on an existing five-module GNC codebase (Project #011); the dual-quaternion kinematic formulation and IEKF/LTV-LQR architecture follow Filipe and Tsiotras [2, 3] and standard CW/Riccati LQR theory [1, 4].

References

- [1] W. H. Clohessy and R. S. Wiltshire, “Terminal Guidance System for Satellite Rendezvous,” *Journal of the Aerospace Sciences*, Vol. 27, No. 9, 1960, pp. 653–658.
- [2] N. Filipe and P. Tsiotras, “Adaptive Position and Attitude-Tracking Controller for Satellite Proximity Operations Using Dual Quaternions,” *Journal of Guidance, Control, and Dynamics*, Vol. 38, No. 4, 2015, pp. 566–577.
- [3] N. Filipe, M. Kontitsis, and P. Tsiotras, “Extended Kalman Filter for Spacecraft Pose Estimation Using Dual Quaternions,” *Journal of Guidance, Control, and Dynamics*, Vol. 38, No. 9, 2015, pp. 1625–1641.
- [4] F. L. Lewis, D. Vrabie, and V. L. Syrmos, *Optimal Control*, 3rd ed., Wiley, Hoboken, NJ, 2012.
- [5] W. Fehse, *Automated Rendezvous and Docking of Spacecraft*, Cambridge University Press, Cambridge, U.K., 2003.
- [6] A. D. Ames, S. Coogan, M. Egerstedt, et al., “Control Barrier Functions: Theory and Applications,” *2019 18th European Control Conference (ECC)*, IEEE, 2019, pp. 3420–3431.

- [7] J. M. Picone, A. E. Hedin, D. P. Drob, and A. C. Aikin, "NRLMSISE-00 Empirical Model of the Atmosphere: Statistical Comparisons and Scientific Issues," *Journal of Geophysical Research: Space Physics*, Vol. 107, No. A12, 2002.
- [8] European Space Agency, "Ariane 5 Storable Propellant Stage (EPS)," https://www.esa.int/Enabling_Support/Space_Transportation/Launch_vehicles/Ariane_5_Storable_Propellant_Stage_EPS, accessed 2026.